
When History Matters: Benchmarking Recurrent RL Agents on Memory Tasks

Anh Khoa Pham
khoa.pham@stud.uni-hannover.de

Quang Ky Anh Nguyen
ky.nguyen@stud.uni-hannover.de

1 Motivation

Standard PPO assumes the Markov property, but many environments are partially observable: the current observation alone is insufficient to act optimally, leading to state aliasing. In MiniGrid-Memory, the agent must recall an object seen earlier to select the correct one at a later fork, a memoryless agent can only guess.

We address this by equipping PPO with a recurrent encoder that maintains a history of past observations. We further ask whether a Transformer encoder handles longer histories better than LSTM, testing this via a reduced observation box ablation.

Research Question: Does equipping PPO with recurrent memory recover performance in partially observable environments, and does replacing LSTM with a Transformer encoder provide an advantage when longer observation histories are required?

2 Related Topics

PPO and DQN, introduced in the RL course, operate under the MDP assumption. LSTM and GRU layers can maintain temporal context across time steps, making them natural candidates for POMDPs. Hausknecht et al. pioneered this combination with DRQN [1], and Pleines et al. (2022) extended it to PPO, introducing RPPO with LSTM/GRU backbones for partially observable environments [2].

3 Experiments

3.1 Environment

We adopt the Memory MiniGrid environment, where the agent begins in a left room with a 7×7 observation field. The task is to identify an object (green key) in the initial room, then proceed to a fork on the right to select the matching object. This instantiates a POMDP: task completion depends on the agent remembering the object’s features across time steps. Reward is sparse, provided only upon task success. As shown in Fig. 1, we start with MiniGrid-MemoryS7-v0 and plan to scale to MiniGrid-MemoryS9-v0 or MiniGrid-MemoryS11-v0 if preliminary results are favorable.

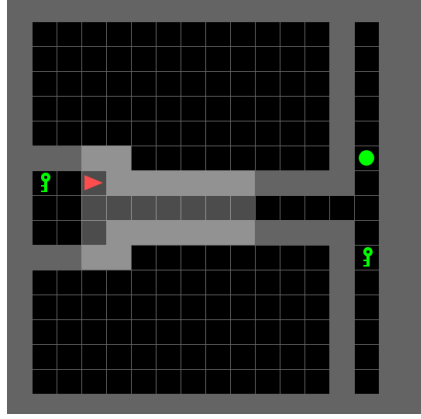


Figure 1: An example of Memory Minigrid enviroment

3.2 Baselines

We include MLP PPO as a memoryless baseline. Since it conditions only on the current observation without any recurrent state, it cannot resolve state aliasing and can only guess when the task requires recalling past observations.

3.3 RPPO

Following Pleines et al., we employ a CNN1D encoder combined with an LSTM layer as the backbone, shared between the actor and critic, with two separate output heads for policy and value. However, we still need to determine through experiments whether sharing the encoder is beneficial or whether splitting it into two independent models yields better performance.

Sequence Padding: Since episodes have variable lengths until termination, we pad all sequences to a uniform length to satisfy LSTM’s input requirements. Embeddings corresponding to padded positions are masked and excluded from loss computation across all three PPO losses (policy loss, value loss, and entropy bonus) [2].

Masking Example: Given batch size 2:

$$\text{Episode 1: } [t_1, t_2, t_3, \text{PAD}, \text{PAD}] \quad (1)$$

$$\text{Episode 2: } [t_1, t_2, t_3, t_4, t_5] \quad (2)$$

After LSTM encoding and masking:

$$\text{Episode 1: } [0.8, 0.6, 0.4, 0.0, 0.0] \quad (3)$$

$$\text{Episode 2: } [0.5, 0.3, 0.7, 0.4, 0.6] \quad (4)$$

The effective batch contains 8 steps (3 from Episode 1, 5 from Episode 2). The remaining PPO loss terms remain unchanged.

3.4 Ablation Study

3.4.1 Transformer PPO

We replace the LSTM layer with a fully connected layer followed by a Transformer encoder, keeping all other architectural choices identical to RPPO. Unlike LSTM, the Transformer attends directly over the full observation sequence without compressing history into a fixed-size hidden state.

3.4.2 Reduced Observation Box (Optional)

LSTM must compress the entire history into a fixed-size hidden state, which becomes a bottleneck for long sequences. Reducing the observation box from 7×7 to 5×5 or 3×3 narrows the agent’s

field of view, forcing more steps to solve the environment and thus producing longer sequences. This setting is designed to expose the LSTM bottleneck and test whether the Transformer handles longer dependencies more effectively.

3.5 Benchmarking

We benchmark all agents on MiniGrid-MemoryS7-v0 with the default 7×7 observation box. Table 1 summarizes our expected outcomes. MLP PPO, being memoryless, cannot resolve state aliasing and is expected to achieve low accumulated return G_t . Both LSTM PPO and Transformer PPO maintain a history of past observations and are expected to solve the task, yielding high G_t .

Table 1: Expected accumulated return G_t on the default 7×7 observation setting.

Algorithm	Expected G_t
MLP PPO	Low
LSTM PPO	High
Transformer PPO	High

Optionally, if time permits, we evaluate on a reduced 5×5 observation box to stress-test the memory mechanisms under longer sequences. We expect LSTM PPO to struggle due to its fixed-size hidden state bottleneck, while Transformer PPO should maintain high G_t thanks to its attention over the full sequence. Table 2 summarizes the expected outcomes.

Table 2: (Optional) Expected accumulated return G_t on the reduced 5×5 observation setting.

Algorithm	Expected G_t
LSTM PPO 5×5	Low
Transformer PPO 5×5	High

4 Timeline

Our project timeline is as follows:

- Literature review: 3 days
- Implementation: 4 days
- Running code: 1 week
- Reporting, visualization, and poster design: 2 days

References

- [1] M. J. Hausknecht and P. Stone, “Deep recurrent q-learning for partially observable mdps.” in *AAAI fall symposia*, vol. 45, 2015, p. 141.
- [2] M. Pleines, M. Pallasch, F. Zimmer, and M. Preuss, “Generalization, mayhems and limits in recurrent proximal policy optimization,” *arXiv preprint arXiv:2205.11104*, 2022.